pyOASIS

Philippe Gambron, Rupert Ford, Andrea Piacentini

Nov 17, 2020

CONTENTS:

CHAPTER

ONE

INTRODUCTION

pyOASIS is a Python wrapper for OASIS written using ctypes and ISO C bindings to Fortran. It provides an object-oriented interface to OASIS. This allows users to write and couple models written in Python or to couple models written in Python with models written in Fortran.

It is part of the distribution of OASIS. See http://www.cerfacs.fr/oa4web/oasis3-mct_4.0/oasis3mct_UserGuide.pdf for more information about obtaining it.

CHAPTER

TWO

INSTALLATION

The sections between brackets are necessary only when Cartopy plots are required, for the examples 10 and 11.

2.1 Under GNU/Linux

2.1.1 Prerequisites

- A Fortran and C compiler suite (tested with version 18 of the Intel compilers as well as versions 7.3, 8.3 and 9 of the GNU compilers).
- An MPI library (tested with version 18 of the Intel MPI Library and versions 4.0.1 and 4.0.5 of OpenMPI)
- NetCDF 4
- Python 3 with mpi4py and NumPy
- [Matplotlib]
- [GEOS (Geometry Engine, Open Source): package libgeos-dev under Debian or Ubuntu]
- [proj: package libproj-dev under Debian or Ubuntu]

2.1.2 Installation

- Obtain OASIS (refer to OASIS User Guide for details).
- Change directory to ${OASIS_ROOT}/util/make_dir.
- Create your own make.inc file based on make.intel_davinci, make.gfortran_openmpi_linux or make.bindings.
- Build and install OASIS and pyOASIS:

```
make -f TopMakefile realclean
make -f TopMakefile pyoasis
```

- Append the lines displayed at the end of the compilation to your .bashrc file or, alternatively, before using pyOASIS, source the script mentioned there.

2.1.3 [Extra software]

- Create a virtual environment:

```
export VENVDIR=~/INSTALL/PY_ENV/PyO   directory containing the virtual environment
python3 -m venv ${VENVDIR}
source ${VENVDIR}/bin/activate
```

- Install software:

```
pip install --upgrade pip
pip uninstall shapely
pip install shapely --no-binary shapely
pip install cartopy
pip install pykdtree
```

The uninstall steps are needed only if a previous version of the software was already there. The last package is not necessary. It is only used for optimisation.

2.2 Under macOS

2.2.1 Prerequisites

```
brew install gcc
brew install openblas
brew install openmpi
[brew install geos]
[brew install proj]
```

(This has been tested with gcc 10.2.0.)

2.2.2 Installation

Same as under GNU/Linux. See previous section.

2.2.3 Virtual Python environment

- Create a virtual environment:

  Same as under GNU/Linux, see previous section.

- Install packages:

```
pip install mpi4py
pip uninstall numpy
pip cache remove numpy
OPENBLAS="$(brew --prefix openblas)" pip install --global-option=build-ext numpy
pip install netcdf4
[pip uninstall scipy]
[pip cache remove scipy]
[pip install --global-option=build-ext scipy]
[pip uninstall shapely]
[pip install shapely --no-binary shapely]
```

(continues on next page)

(continued from previous page)

```
[pip install matplotlib]
[pip install cartopy]
[pip install pykdtree]
```

The uninstall steps are needed only if a previous version of the software was already there. The last package is not necessary. It is only used for optimisation.

2.2.4 Set up the environment

At the end of your .bashrc,

```
export TMPDIR=/tmp
export OMPI_MCA_mca_base_env_list=LD_LIBRARY_PATH=${PYOASIS_ROOT}/lib
```

2.3 Tests

pyOASIS can be tested by issuing, in the directory ${OASIS_ROOT}/pyoasis:

```
make test
```

This will test the Python wrapper itself as well as running examples using OASIS. It requires pytest.

2.4 Documentation

If pyOASIS is modified, this document can be regenerated, using Sphinx, by typing the following command in the directory ${OASIS_ROOT}/pyoasis:

```
make doc
```

CHAPTER

THREE

USING PYOASIS

3.1 Code structure

The source code of pyOASIS is in the directory `src`. First, the OASIS Fortran code is wrapped in Fortran using ISO-C bindings. The corresponding source files are in the subdirectory `src/fortran_isoc`. The file names are the same as the corresponding ones in the original source code but ending in `_iso.F90`. Subsequently, the Fortran with ISO-C bindings is wrapped in C. This time, the source code is in `src/c`. As before, the names of the files are the same as the corresponding Fortran ones, but ending in `_iso.c`. Finally, the C is wrapped in Python in the directory `src/python`. A low-level wrapper is made using the same filenames as the Fortran ones but ending in `.py`. A higher level object-oriented wrapper is contained in the file `pyoasis.py`. This higher level wrapper provides the pyOASIS interface.

3.2 Creating a component

In pyOASIS, components are instances of the **Component** class. To initialise a component, its name has to be supplied.:

```
import pyoasis
component_name = "component"
comp = pyoasis.Component(component_name)
```

It is also possible to provide an optional `coupling_flag` argument which defaults to coupled.:

```
import pyoasis
component_name = "component"
coupling_flag = True
comp = pyoasis.Component(component_name, coupling_flag)
```

3.3 Using MPI

OASIS couples models which communicate using MPI. By default, the **Component** class will set up MPI internally and provides methods to get access to information such as rank and number of processes. In this case, the global communicator used is MPI.COMM_WORLD.:

```
import pyoasis

comp = pyoasis.Component("component")

```

(continues on next page)

(continued from previous page)

```
print("Hello world from process " + str(comp.localcomm.rank)
      + " of " +  str(comp.localcomm.size))
```

If the user wants to use his or her own communicator, this can be passed to the **Component** class through the communicator optional argument. This should be created with `mpi4py`.

```
import pyoasis
from mpi4py import MPI

comm = MPI.COMM_WORLD

component_name = "component"
coupling_flag = True
comp = pyoasis.Component(component_name, coupling_flag, comm)
```

3.4 Creating a partition

The data can be partitioned in various ways. These correspond to the **SerialPartition**, **ApplePartition**, **BoxPartition**, **OrangePartition** and **PointsPartition** classes which are inherited from the **Partition** abstract class.

The simplest situation is the serial partitioning where all the data is held by a single process and only the number of points has to be specified.

```
n_points = 16
serial_partition = pyoasis.SerialPartition(n_points)
```

In the case of the apple partitioning, each process contains a segment of a linear domain. To initialise such a partitioning, an offset has to be supplied for each rank as well as the number of data points that will be stored locally. The following example, if run with 4 processes, will produce 4 consecutive local segments containing 4 data points.

```
component_name = "component"
comp = pyoasis.Component(component_name)
rank = comp.localcomm.rank
size = comp.localcomm.size
n_points = 16
local_size = int(n_points/size)
offset = rank * local_size
partition = pyoasis.ApplePartition(offset, local_size)
```

When we use the box partitioning, a 2-dimensional domain is split into several rectangles. The global offset, local extents in the x and y directions and the global extent in the x direction have to be supplied to the constructor. The global offset is the index of the corner of the local rectangle. For example, we can split a 4x4 square domain into 4 2x2 parts with the following code that will have to be executed using 4 processes. The offset is computed from the global and local domain sizes as well as the rank.

```
rank = comp.localcomm.rank
n_global_points_per_side = 4
n_partitions_per_side = 2
local_extent = n_global_points_per_side / n_partitions_per_side
i_partition_x = rank / n_partitions_per_side
i_partition_y = rank % n_partitions_per_side
global_offset =   i_partition_x * n_global_points_per_side * local_extent
                + i_partition_y * local_extent
```

(continues on next page)

(continued from previous page)

```
global_extent_x = n_global_points_per_side
partition = pyoasis.BoxPartition(global_offset, local_extent, local_extent,
                                 global_extent_x)
```

The orange partitioning consists of several segments of a linear domain. As a consequence, a list of offsets and local sizes have to be provided. In this example, each process contains 2 consecutive segments of 2 points.

```
rank = comp.localcomm.rank
n_segments_per_rank = 2
n_points_per_segment = 2
offset_beginning = rank * n_segments_per_rank * n_points_per_segment
offset = [offset_beginning, offset_beginning + n_points_per_segment]
extents = [n_points_per_segment, n_points_per_segment]
partition = pyoasis.OrangePartition(offsets, extents)
```

The last type of partitioning is points, where we have to specify, in a list, the global indices of the points stored by the process.

```
global_indices=[0, 1, 2, 3]
partition = pyoasis.PointsPartition(global_indices)
```

See the OASIS documentation for more information about the various types of partitioning.

3.5 Initialising the data

The data is handled by the class **Var**. Its constructor requires the name, as is appears in the `namcouple` file, the partition, the rank with which we wish to communicate and a flag indicating whether the data is incoming or outgoing. The latter is an enumerated type and can have the values `pyoasis.OasisParameters.OASIS_OUT` or `pyoasis.OasisParameters.OASIS_IN`. In the following example, we wish to send data to a process having the rank 1 and we use a partition that was previously created.:

```
data_name = "name"
variable = pyoasis.Var(data_name, partition,
                       pyoasis.OasisParameters.OASIS_OUT)
```

In the case of the receiving model, the code is:

```
data_name = "name"
variable = pyoasis.Var(data_name, partition,
                       pyoasis.OasisParameters.OASIS_IN)
```

We must end the definition of the component by calling the `enddef()` method.

```
comp.enddef()
```

However this must be done only once the partitioning and the variable data have been initalised.

3.6 Sending and receiving data

pyOASIS expects data to be provided as a **pyoasis.asarray** object. This is a Numpy array but ordered in the Fortran way. In C, multidimensional arrays store data in row-major order where contiguous elements are accessed by incrementing the rightmost index while varying the other indices will correspond to increasing strides in memory as we use indices further towards the left. By default, Numpy arrays use that ordering as well. Fortran, on the other hand, uses column-major order. In that case, contiguous elements are accessed by incrementing the leftmost index. **pyoasis.asarray** objects use the same ordering as Fortran. As a consequence, it is not necessary to transform data in order to use it in the OASIS Fortran library.

```
field = pyoasis.asarray(range(n_points))
```

We must also associate a time to the data. If the receiving model specifies that same time as this model then it receives the data.

```
date = int(0)
```

The data is sent with the following function.

```
variable.put(date, field)
```

Conversely, it is received with the function

```
variable.get(date, field)
```

It expects data carrying the same date and will fill the **pyoasis.asarray** object.

Finally, the coupling is terminated in the destructor of the component.

3.7 Exceptions

When an error occurs in OASIS and the code coupler returns an error code, an **OasisException** is raised and when an error is caught by the pyOASIS wrapper, such as an incorrect parameter or a wrong argument type, a **PyOasisException** is raised.

In the following example, where we attempt to initialise a component, a **PyOasisException** will be raised if the user supplies an empty name or a component of the wrong type. On the other hand, if a problem occurs in OASIS, an **OasisException** is raised.

```
try:
    comp = pyoasis.Component("name")
except (pyoasis.OasisException, pyoasis.PyOasisException) as exception:
    pyoasis.pyoasis_abort(exception)
```

A more complete example involving exceptions can be found in `test/apple_and_orange`, in the files `receiver-orange.py` and `sender-apple.py`. These are used to test pyOASIS with a working example involving two components communicating and can show how exceptions might be handled in a real code. There are cases where OASIS will abort before pyOASIS can raise an exception. This happens, for instance, when the name of the variable data is inconsistent with the contents of the `namcouple` file. However, in such a case, one can rely on the error interception taking place in OASIS which will describe the issue in the log files.

CHAPTER

FOUR

EXAMPLES

4.1 Serial partitions

This example consists in two models, one sending data to another. The sender and receiver start in the same way.

-Import pyOASIS and initialise the MPI communicator

```
import pyoasis
from mpi4py import MPI
comm = MPI.COMM_WORLD
```

-Initialisation of the component

```
comp = pyoasis.Component(component_name)
```

-Initialisation of the serial partition

```
partition = pyoasis.SerialPartition(n_points)
```

-From this point, the sender and the receiver start to differ. In the sender, the variable data is initialised by

```
variable = pyoasis.Var("FSENDOCN", partition,
                       pyoasis.OasisParameters.OASIS_OUT)
```

whereas, in the receiver, we have

```
variable = pyoasis.Var("FRECVATM", partition,
                       pyoasis.OasisParameters.OASIS_IN)
```

where the last flag is instead `pyoasis.OasisParameters.OASIS_IN` to indicate that, in this case, the data will be incoming.

In both scripts, the initialisation of the component ends by

```
comp.enddef()
```

In the sender, the data is subsequently transmitted by

```
time_in_the_model = int(0)
field = pyoasis.asarray(range(n_points))
variable.put(time_in_the_model, field)
```

while, in the receiver, it is recovered by

```
time_in_the_model = int(0)
field = pyoasis.asarray(numpy.zeros(n_points))
variable.get(time_in_the_model, field)
```

The time in the model must be the same for the two components.

The full source code as well as the namcouple file and a script to run this example are in the directory `pyoasis/examples/1-serial/python`.

4.2 Apple and orange partitions

In this example, both models run as several processes. The beginning of the code is identical to the previous example. We will highlight only the differences. In the sender, the data is split according to the apple partitioning

```
partition = pyoasis.ApplePartition(offset, local_size)
```

whereas the receiver uses the orange partitioning.

```
partition = pyoasis.OrangePartition(offsets, extents)
```

In both cases, the offsets and sizes of the local part of the data have to be specified. Each process subsequently transmits and receives its share of the data as previously. In the sender, we have

```
date = int(0)
field = pyoasis.asarray(numpy.zeros(local_size))
for i in range(local_size):
    field[i] = offset + i
variable.put(date, field)
```

while, in the receiver,

```
date = int(0)
field = pyoasis.asarray(numpy.zeros(extent))
variable.get(date, field)
```

The complete example can be found in `examples/6-apple_and_orange/python`.

4.3 Fortran and Python interoperability

In order to illustrate the possibility to couple models written in Python and in Fortran, we repeat the previous example where, this time, the sender has been written in Fortran. The receiver is the same as above.

The sender consists in an analogous sequence.

-Initialisation of the component

```
call oasis_init_comp(comp_id, comp_name, kinfo)
```

-Creation of the coupling communicator from the one used by the component

```
call oasis_get_localcomm(local_comm, kinfo)
call oasis_create_couplcomm(1, local_comm, coupl_comm, kinfo)
```

-Initialisation of the apple partition with the relevant offset and local size

```
part_params=(/1, offset, local_size/)
call oasis_def_partition(part_id, part_params, kinfo)
```

-Creation of the variable data

```
var_nodims=(/1, 1/)
var_actual_shape=1
call oasis_def_var(var_id, var_name, part_id, var_nodims, OASIS_OUT,
                   var_actual_shape, OASIS_REAL, kinfo)
```

-End of the definition of the component

```
call oasis_enddef(kinfo)
```

-Transmission of the local part of the data to the other component

```
call oasis_put(var_id, date, field, kinfo)
```

-End of the coupling

```
call oasis_terminate(kinfo)
```

The complete example can be found in `pyoasis/examples/7-fortran_and_python`.

CHAPTER

FIVE

API REFERENCE

The class **Component** manages a component that will be coupled by OASIS. The data can be split in various ways corresponding to the classes **SerialPartition**, **ApplePartition**, **BoxPartition**, **OrangePartition** and **PointsPartition** (see the OASIS documentation for more details). Finally the data is handled by the class **Var**.

class pyoasis.**OasisException**(*text*, *error*)

class pyoasis.**PyOasisException**(*text*)

pyoasis.**pyoasis_abort**(*exception*)

Terminates cleanly the execution of OASIS and pyOASIS while dsiplaying an error message and the context in which the exception was raised.

Parameters **exception** (*Exception*) – exception to be handled

Raises ***PyOasisException*** – if an incorrect parameter is supplied

class pyoasis.**Component**(*name*, *coupled=True*, *communicator=<mpi4py.MPI.Intracomm object>*)

Component that will be coupled by OASIS

Parameters

- **name** (*str*) – name of the component
- **coupled** (*bool*) – whether the component will be coupled (default: True)
- **communicator** (*mpi4py.MPI.Intracomm*) – global MPI communicator (default: MPI.COMM_WORLD)

Raises

- ***OasisException*** – if OASIS is unable to initialise the component
- ***PyOasisException*** – if an incorrect parameter is supplied

create_couplcomm(*icpl*)

Creates a coupling communicator

Parameters **icpl** (*int*) – coupling switch (1 coupled process, 0 not coupled)

Returns error code

Return type int

Raises

- ***OasisException*** – if OASIS is unable to create the coupling communicator
- ***PyOasisException*** – if an incorrect parameter is supplied

property debug_level

Returns the currently set debug level

Returns debug level

Return type int

Raises ***OasisException*** – if OASIS is unable to get the debug level

static enddef()
Ends the initialisation of the component.

Raises ***OasisException*** – if OASIS is unable to end the initialisation

get_intercomm(*compname*)

Parameters **compname** (*string*) – name of the other component in the intercommunicator

Returns the intercommunicator

Return type MPI.communicator

Raises

- ***OasisException*** – if OASIS is unable to create the intercommunicator
- ***PyOasisException*** – if an incorrect parameter is supplied

get_intracomm(*compname*)

Parameters **compname** (*string*) – name of the other component in the intracommunicator

Returns the intracommunicator

Return type MPI.communicator

Raises

- ***OasisException*** – if OASIS is unable to create the intracommunicator
- ***PyOasisException*** – if an incorrect parameter is supplied

property name

Returns the name of the component

Return type string

set_couplcomm(*couplcomm*)
Sets the coupling communicator :param MPI.communicator couplcomm: coupling communicator

Returns error code

Return type int

Raises

- ***OasisException*** – if OASIS is unable to set the coupling communicator
- ***PyOasisException*** – if an incorrect parameter is supplied

class pyoasis.**SerialPartition**(*size*, *global_size=-1*, *name=''*)
Serial partition

Parameters

- **size** (*int*) – number of points in the partition
- **global_size** (*int*) – global size of the grid (optional)
- **name** (*str*) – name of the partition (optional)

Raises

- ***OasisException*** – if OASIS is unable to initialise the partition
- ***PyOasisException*** – if an incorrect parameter is supplied

class pyoasis.**AppelePartition**(*offset*, *size*, *global_size=-1*, *name=''*)
Apple partition

Parameters

- **offset** (*int*) – offset according to the global index
- **size** (*int*) – number of points in the partition
- **global_size** (*int*) – global size of the grid (optional)
- **name** (*str*) – name of the partition (optional)

Raises

- ***OasisException*** – if OASIS is unable to initialise the partition
- ***PyOasisException*** – if an incorrect parameter is supplied

class pyoasis.**BoxPartition**(*global_offset*, *local_extent_x*, *local_extent_y*, *global_extent_x*, *global_size=-1*, *name=''*)
Box partition

Parameters

- **global_offset** (*int*) – offset according to the global index
- **local_extent_x** (*int*) – extent in the x direction of the local partition
- **local_extent_y** (*int*) – extent in the y direction of the local partition
- **global_extent_x** (*int*) – global extent in the x direction
- **global_size** (*int*) – global size of the grid (optional)
- **name** (*str*) – name of the partition (optional)

Raises

- ***OasisException*** – if OASIS is unable to initialise the partition
- ***PyOasisException*** – if an incorrect parameter is supplied

class pyoasis.**OrangePartition**(*offsets*, *extents*, *global_size=-1*, *name=''*)
Orange partition

Parameters

- **offsets** (*list of integers*) – list of offsets according to the global index
- **extents** (*list of integers*) – list of the partition extents
- **global_size** (*int*) – global size of the grid (optional)
- **name** (*str*) – name of the partition (optional)

Raises

- ***OasisException*** – if OASIS is unable to initialise the partition
- ***PyOasisException*** – if an incorrect parameter is supplied

class pyoasis.**PointsPartition**(*global_indices*, *global_size=-1*, *name=''*)
Points partition

Parameters

- **global_indices** (*list of integers*) – list containing the global indices of the points in the partition
- **global_size** (*int*) – global size of the grid (optional)
- **name** (*str*) – name of the partition (optional)

Raises

- ***OasisException*** – if OASIS is unable to initialise the partition
- ***PyOasisException*** – if an incorrect parameter is supplied

pyoasis.**OasisParameters**()
Enumeration of parameters used by OASIS (values: OASIS_OUT, OASIS_IN)

pyoasis.**asarray**(*data*, *dtype=<class 'numpy.float64'>*)
Numpy array of double precision floating point numbers in Fortran ordering

Parameters

- **data** – any object that can be used to initialise a numpy array
- **dtype** – the numpy datatype of the returned array

Raises ***PyOasisException*** – if a Numpy array cannot be initialised

class pyoasis.**Var**(*name*, *partition*, *inout*, *bundle_size=1*)
Variable data

Parameters

- **name** (*string*) – name of the variable data [cdport in OASIS]
- **inout** (*pyoasis.OasisParameter*) – flag indicating whether the data is outgoing or ingoing
- **bundle_size** (*int*) – size of a bundle of fields

Raises

- ***OasisException*** – if OASIS is unable to initialise the variable data
- ***PyOasisException*** – if an incorrect parameter is supplied

property cpl_freqs

Returns the coupling periods

Return type array of integers

Raises ***OasisException*** – if OASIS is unable to obtain the

coupling periods

get(*time*, *field*)
Gets data from another model.

Parameters

- **time** (*int*) – model time (in seconds) [kstep in OASIS]
- **field** (*pyoasis.asarray*) – data

Raises

- ***OasisException*** – if OASIS is unable to receive data from the other component
- ***PyOasisException*** – if an incorrect parameter is supplied

property name

Returns name of variable data

Return type string

put(*time*, *field*, *write_restart=False*)
Sends data to another model.

Parameters

- **time** (*int*) – model time (in seconds) [kstep in OASIS]
- **field** (*pyoasis.asarray*) – data
- **write_restart** (*bool*) – optional flag for writing a restart file

Raises

- ***OasisException*** – if OASIS is unable to send data to the other component
- ***PyOasisException*** – if an incorrect parameter is supplied

put_inquire(*time*)

Parameters **time** (*int*) – model time (in seconds) [msec in OASIS]

Returns return value expected at a specified time

for a given variable

Return type pyoasis.OasisParameters

Raises ***OasisException*** – if OASIS is unable to obtain

the return code

Raises ***PyOasisException*** – if an incorrect parameter is

supplied

Grids are managed by the *Grid* class.

class pyoasis.**Grid**(*cgrid*, *nx_global*, *ny_global*, *lon*, *lat*, *partition=None*)
Grid declared via pyoasis API and written to netcdf files grids.nc masks.nc areas.nc

Parameters

- **cgrid** (*string*) – grid name (should be shorter than 64 char)
- **nx_global** (*int*) – global (non distributed) x size of the grid
- **ny_global** (*int*) – global (non distributed) y size of the grid
- **lon** (*numpy.array*) – longitudes of the cell centres (2D array)
- **lat** (*numpy.array*) – latitudes of the cell centres (2D array)
- **partition** (*type*) – optional partition object for distributed grids

Raises ***PyOasisException*** – if the name is empty

set_area(*area*)
Set area values

Parameters **area** (*array of double precision floating numbers*) – mask array

set_corners(*clo*, *cla*)
Sets corner latitudes and longitudes

Parameters

- **clo** (*array of double-precision floating numbers*) – array longitudes
- **cla** (*array of double-precision floating numbers*) – array longitudes

set_mask(*mask*, *companion=None*)
Sets integer mask values

Parameters

- **mask** (*array of integer numbers*) – mask array
- **companion** (*str*) – companion

write()
Writes the grid

CHAPTER

SIX

CORRESPONDENCE WITH THE OASIS INTERFACE

These tables show the correspondence between the functions described in the OASIS user guide and their analogue in pyOASIS. All the functions have been implemented with their full interface except `put` which, in the case of pyOASIS, accepts only a single field.

All these functions are tested, at the level of the Python wrapper, using pytest whereas they are tested, while running OASIS, by the examples. All these tests cans be executed by the command described in section 2.3.

6.1 Component

OASIS	pyoasis.Component.
oasis_init_comp(comp_id, comp_name, ierror, coupled, comm_world)	**__init__(name, coupled=True,** communicator=MPI.COMM_WORLD)
oasis_terminate(ierror)	__del__()
oasis_create_couplcomm(icpl, localcomm, couplcomm, kinfo)	create_couplcomm(icpl)
oasis_set_couplcomm(couplcomm, kinfo)	set_couplcomm(couplcomm)
oasis_get_intracomm(newcomm, cdnam, kinfo)	get_intracomm(compname)
oasis_get_intercomm(newcomm, cdnam, kinfo)	get_intercomm(compname)
oasis_enddef(ierror)	enddef()
oasis_get_local_comm(local_comm, | localcomm ierror) |	
oasis_get_debug(debugvalue)	debug_level
oasis_set_debug(debugvalue)	debug_level

6.2 Partition

OASIS	pyoasis.
oasis_def_partition(ilpart_id, igparal, ierror, isize, name)	**SerialPartition(size,** global_size=-1, name="") **ApplePartition(offset, size,** global_size=-1, name="") **BoxPartition(global_offset,** local_extent_x, local_extent_y, global_extent_x, global_size=-1, name="") **OrangePartition(offsets, extents,** global_size=-1, name="") **PointsPartition(global_indices,** global_size=-1, name="")

6.3 Var

OASIS	pyoasis.var.
oasis_def_var(var_id, name, il_part_id, var_nodims, kinout, var_actual_shape, vartype, ierror)	**__init__(name, partition, inout,** bundle_size=1)
oasis_put(varid, date, fld1, info, fld2, fld3, fld4, fld5, write_restart)	**put(time, field,** write_restart=False)
oasis_get(varid, date, fld, info)	get(time, field)
oasis_put_inquire(varid, date, kinfo)	put_inquire(time)
oasis_get_ncpl(varid, ncpl, kinfo)	len(cpl_freqs)
oasis_get_freqs(varid, mop, ncpl, cplfreqs, kinfo)	cpl_freqs

6.4 Grid

OASIS	pyoasis.grid.
oasis_start_grids_writing(flag) oasis_write_grid(cgrid, nx_global, ny_global, lon, lat, il_partid)	**__init__(cgrid, nx_global,** ny_global, lon, lat, partition=None)
oasis_write_corner(cgrid, nx_global, ny_global, nc, clon, clat, il_partid)	set_corners(clo, cla)
oasis_write_area(cgrid, nx_global, ny_global, area, il_partid)	set_area(area)
oasis_write_mask(cgrid, nx_glo, ny_glo, mask, part_id, companion)	**set_mask(mask,** companion=None)
oasis_write_frac(cgrid, nx_glo, ny_glo, frac, part_id, companion)	**set_frac(frac,** companion=None)
oasis_terminate_grids_writing()	write()

6.5 Utilities

OASIS	pyoasis.
oasis_abort(compid, routinename, abortmessage, rcode)	**oasis_abort(component_id,** routine, message, filename, line, error) pyoasis_abort(exception)

CHAPTER

SEVEN

ACKNOWLEDGMENTS

This work has been financed by the ISENES3 project which has received funding from the European Union's Horizon 2020 research and innovation programme under grant agreement No 824084.

CHAPTER

EIGHT

INDEX AND SEARCH

- genindex
- search

INDEX